Concurrency Keywords and Features in Major Languages

Overview

This document compares the concurrency-related keywords and features in several programming languages and Ada versions. It includes keyword-level differences and high-level concurrency support.

Concurrency Keyword Comparison Table

Feature / Keyword	C++	Python	Java	Rust	Ada83	Ada95	Ada2005	Ada2012	Ada2022
Thread creation	std::thread	threading.Thread	Thread, Runnable	std::thread	task	task	task	task	task
Shared data protection	std::mutex	Lock, RLock	synchronized	Mutex<T>	N/A	protected	protected	protected	protected
Atomic ops	std::atomic	Lock (manual)	AtomicInteger	Atomic* types	N/A	N/A	atomic (pkg)	hints	RT support
Task communication	cond_var, future	Queue, Event	wait, notify	channel, Sender	entry, accept	entry, accept	entry/accept	entry/requeue	enhanced rendezvous
Async/await model	co_await (C++20)	async, await	CompletableFuture	async, .await	No	No	No	limited	async/await, parallel
Coroutines	co_yield	yield	Project Loom	native	No	No	No	No	work in progress
Parallel loops	Manual	multiprocessing	ForkJoinPool	rayon::par_iter	No	No	Limited	parallel	enhanced
Tasks as objects	thread+lambda	subclass Thread	Runnable, Callable	FnOnce, closures	task	task type	task type	task type	task+extensions
Channels/Queues	via libraries	queue.Queue	BlockingQueue	mpsc::channel	N/A	N/A	N/A	limited	Ada.Real_Time

Language-Specific Highlights

C++:

- Threads via std::thread, manual sync

- C++20 coroutines

Python:

- threading, asyncio, GIL limits real concurrency

Java:

- Thread model, synchronized, CompletableFuture, Loom (upcoming)

Rust:

- Ownership-based safety, channels, async/await

Ada83:

- task, entry, accept for concurrency

Ada95:

- Adds protected types for safe shared access

Ada2005:

- Better real-time support, Ravenscar profile

Ada2012:

- Contracts, requeue, expressive synchronization

Ada2022:

- parallel, async/await, task groups, multicore support